

Part D – apply the deviation during analysis (REQ 12)

Until now a deviation is stored and displayed but changes nothing: when a flight is downloaded, the alarm analysis still reads the configuration thresholds. Six steps make the analysis use the deviated values instead, for sessions inside the deviation's date range.

Where it goes, and why there

HealthMonitoringAnalyzer calls RedAmberAlarmAnalyzer.computeRedAmberAlarms, and that is the only place an indicator value is compared against a threshold to raise an alarm. Everything else in that class handles grey alarms, sensor anomalies or acquisition rate. So the whole of REQ 12 lands in one method.

mgs-download has no database access of its own – it receives everything through IHealthHistoryProvider, whose implementation sits in mgs-services-impl with an EntityManager. The deviations arrive the same way, which keeps the analyzer free of DAO dependencies.

What a deviation overrides	Used in the analyzer for
Threshold value	The comparison itself
Threshold level	The signum – whether an alarm is above or below
Successive points	How many points in a row before an alarm is raised
Work card	Stamped on the alarm that is created

A deviation is a patch, not a replacement. Since Benoit's decision it stores only the fields that differ from the configuration, so each override below is guarded by a null check. A deviation that changed only the value leaves the level, the successive points and the work card coming from the configuration, which is the intended behaviour.

In plain words

#	What each step does
1	Teach the database how to find every deviation belonging to an aircraft.
2	Add one line to the interface the analysis uses to ask for data.
3	Write the code that actually runs that database query.
4	In the alarm code, before comparing a value to its threshold, check whether a deviation applies – and if it does, use the deviated numbers instead.
5	Write the small method that decides whether a deviation applies to this session, by comparing dates.
6	Make sure the download module is allowed to see the new class.

Steps 1 to 3 fetch the deviations. Steps 4 and 5 use them. Step 6 is a one-minute check.

The rule

A deviation applies to a session when it started on or before that session, and is either still open or was closed on or after it. The date that counts is the session's own data date, not the day it was offloaded.

Deviation	Session dated	Values used
Created 1 Feb, still open	31 Jan	Configuration
Created 1 Feb, still open	4 Feb	Deviated
Created 1 Feb, closed 15 Feb	14 Feb	Deviated
Created 1 Feb, closed 15 Feb	17 Feb	Configuration

On the Flight entity the data date is getHealthStartDate(), falling back to getUsageStartDate() when it is null. Not getLastDownloadDate(), which is the offload date – the requirement is explicit about this, and a flight downloaded weeks later would otherwise be judged against the wrong deviation.

The six steps

Step 1 — IndicatorDeviation – a named query by aircraft

mgs-jpa-model/.../datamodel/health/monitoring/IndicatorDeviation.java

Add a third @NamedQuery beside the two already there. The analysis needs every deviation of the aircraft, open or closed, because an old closed one still applies to an old session.

```
@NamedQuery(name = "findAllDeviationsByAircraft",
    query = "SELECT d FROM IndicatorDeviation d "
        + "JOIN d.indicatorConfiguration ic "
        + "WHERE ic.indicator.id IN ( "
        + "    SELECT i.id FROM Aircraft a JOIN a.healthComponents hmc JOIN hmc.indicators i "
        + "    WHERE a.id = :aircraftId)"
```

If the path from Aircraft to its indicators is named differently, adjust the subquery – RedAmberAlarmAnalyzer walks aircraft.getHealthComponents() then getIndicators().values(), so those are the association names to follow.

Step 2 — IHealthHistoryProvider – one new method

mgs-download/.../download/analysis/health/api/IHealthHistoryProvider.java

The analyzer asks the provider for the deviations, exactly as it already asks for indicator values and the latest configuration.

```
/**
 * BCR-6605 every deviation of the aircraft, active or acknowledged, keyed by
 * indicator id and severity, for example "abc123|RED".
 */
Map<String, IndicatorDeviation> getDeviations();
```

Import com.eurocopter.mgs.datamodel.health.monitoring.IndicatorDeviation.

Step 3 — HealthHistoryProvider – the query

mgs-services-impl/.../service/download/ejb/HealthHistoryProvider.java

Implement it with the EntityManager this class already holds. Loaded once per analysis rather than per threshold, which matters: a configuration has hundreds of indicators.

```
@Override
public Map<String, IndicatorDeviation> getDeviations() {
    TypedQuery<IndicatorDeviation> query =
        manager.createNamedQuery("findAllDeviationsByAircraft", IndicatorDeviation.class);
    query.setParameter("aircraftId", aircraft.getId());

    Map<String, IndicatorDeviation> deviations = new HashMap<>();
    for (IndicatorDeviation deviation : query.getResultList()) {
        String key = deviation.getIndicatorConfiguration().getIndicator().getId()
            + "|" + deviation.getSeverity();
        // Keep the most recently created one when an indicator has been deviated more than once
        IndicatorDeviation existing = deviations.get(key);
        if (existing == null || deviation.getStartDate().after(existing.getStartDate())) {
            deviations.put(key, deviation);
        }
    }
    return deviations;
}
```

The map keeps one deviation per indicator and severity, the most recent. That is a simplification: an indicator deviated, acknowledged, then deviated again has two rows, and a session dated between them should use the first. If that case matters, the map value becomes a list and appliesTo picks from it – worth asking Benoit whether it can happen in practice before adding the complexity.

Step 4 — RedAmberAlarmAnalyzer – look up the deviation

mgs-download/.../download/analysis/health/impl/RedAmberAlarmAnalyzer.java

Three edits in computeRedAmberAlarms.

4.1 At the top of the method, beside the two maps already fetched:

```

Map<Indicator, Map<Flight, HealthAlarm>> indicatorAlarms = new HashMap<>(healthHistoryProvider.getExistingHealthAlarms());
Map<Indicator, HealthAlarm> latestCloseMonitoredAlarms = healthHistoryProvider.getLatestCloseMonitoredAlarms();

// BCR-6605 every deviation of this aircraft, applied below when the session date matches
Map<String, IndicatorDeviation> deviations = healthHistoryProvider.getDeviations();

```

4.2 Inside the threshold loop, immediately after `currentFlight` is in scope and before the signum line. This resolves which deviation, if any, governs this threshold for this session.

```

for (final ThresholdConfiguration thresholdConfiguration : indicatorConfiguration.getThresholdConfigurations()) {

    // ===== BCR-6605 add from here =====
    final IndicatorDeviation deviation = getApplicableDeviation(deviations, indicator,
        thresholdConfiguration.getThreshold().getSeverity(), currentFlight);
    // ===== to here =====

    Integer nbSuccessiveValues = thresholdConfiguration.getNbSuccessivePoints();
    if (deviation != null && deviation.getNbSuccessivePoints() != null) {
        nbSuccessiveValues = deviation.getNbSuccessivePoints(); // BCR-6605
    }
    if (nbSuccessiveValues == null) {
        nbSuccessiveValues = defaultNbSuccessiveValues;
    }
    ...
}

```

4.3 The signum and the threshold value. Replace the block that resolves `thresholdValue`; the deviated value wins over both the fixed value and the learned one.

```

// BCR-6605 the deviated level decides the direction of the comparison
ThresholdLevel level = thresholdConfiguration.getThresholdLevel();
if (deviation != null && deviation.getThresholdLevel() != null) {
    level = ThresholdLevel.valueOf(deviation.getThresholdLevel());
}
int signum = (level == ThresholdLevel.HIGH) ? 1 : -1;

final double thresholdValue;
if (deviation != null && deviation.getThresholdValue() != null) {
    // BCR-6605 an approved deviation replaces the configured value for this session
    thresholdValue = deviation.getThresholdValue();
} else if (thresholdConfiguration instanceof FixedThresholdConfiguration) {
    thresholdValue = thresholdConfiguration.getValue();
} else if (thresholdConfiguration instanceof LearnedThresholdConfiguration) {
    RelearnThresholdRequest relearnRequest = getApplicableRelearnRequest(currentFlight, thresholdConfiguration);
    if (relearnRequest != null) {
        thresholdValue = relearnRequest.getLearnedValue();
    } else {
        final Double defaultValue = thresholdConfiguration.getValue();
        if (defaultValue != null) {
            thresholdValue = defaultValue;
        } else {
            LOGGER.info("Skipping learning threshold '{}': no default value defined",
                thresholdConfiguration.getThreshold().getFunctionalId());
            continue;
        }
    }
} else {
    throw new IllegalStateException();
}

```

A deviation on a learned threshold therefore overrides the learned value for as long as it is active. That follows from the requirement – the customer has an approved departure from the directive – but it is worth confirming with Benoit, since it means a relearn completed during the deviation has no effect until the deviation is closed.

4.4 The work card on the alarm. Two places set it, both inside the alarm creation block further down. Replace each `setWorkcardReference` line:

```

// BCR-6605 the deviated work card, when the deviation changed it
String workcard = thresholdConfiguration.getWorkcard();
if (deviation != null && deviation.getWorkcard() != null) {
    workcard = deviation.getWorkcard();
}
previousAlarm.setWorkcardReference(workcard);

```

Step 5 — RedAmberAlarmAnalyzer – the applicability test

Add this private method at the end of the class. It is the whole of the date rule from page one.

```

/**
 * BCR-6605 the deviation that governs this threshold for this session, or null.
 *
 * A deviation applies when it started on or before the session's data date, and is
 * either still open or was closed on or after it. The date used is the session's own
 * data date, never the download date: a flight offloaded weeks later must still be
 * judged against the deviation that was in force when it was flown.
 */
private IndicatorDeviation getApplicableDeviation(final Map<String, IndicatorDeviation> deviations,
    final Indicator indicator, final Severity severity, final Flight currentFlight) {
    if (deviations.isEmpty()) {
        return null;
    }
    final IndicatorDeviation deviation = deviations.get(indicator.getId() + "|" + severity.name());
    if (deviation == null) {
        return null;
    }

    final Date sessionDate = currentFlight.getHealthStartDate() != null
        ? currentFlight.getHealthStartDate()
        : currentFlight.getUsageStartDate();
    if (sessionDate == null) {
        // Nothing to compare against, so the configuration applies
        return null;
    }

    if (deviation.getStartDate().after(sessionDate)) {
        // The session predates the deviation
        return null;
    }
    if (deviation.getEndDate() != null && deviation.getEndDate().before(sessionDate)) {
        // The deviation was already closed when this session was flown
        return null;
    }
    return deviation;
}

```

Imports: java.util.Date and com.eurocopter.mgs.datamodel.health.monitoring.IndicatorDeviation. Severity and Flight are already imported in this class.

Step 6 — mgs-download – check the dependency

mgs-download/META-INF/MANIFEST.MF

IndicatorDeviation lives in mgs-jpa-model, which this bundle already uses for ThresholdConfiguration and the rest, so the package com.eurocopter.mgs.datamodel.health.monitoring should already be imported. Confirm it, and add it in the Dependencies tab if the import does not resolve.

How to test

Part D is the first piece that changes what MGS actually does. Everything before it stored and displayed a deviation; this one makes a deviation stop an alarm. So the test is simply: does the alarm go away, and does it come back.

Before you start

Rebuild mgs-jpa-model, mgs-download, mgs-services-impl and the client, then redeploy.

Find an indicator that raises an alarm today. The Health tree shows red or amber ones, or look in Alarm History. Note the threshold value it is breaching.

Test 1 – the alarm stops

#	Do this	Expect
1	Create a deviation on that indicator. Set the Value well above anything the data reaches, add a TE, press Apply and OK.	The row turns green
2	Download a flight.	No alarm on that indicator
3	Run: SELECT severity, date FROM public.alarm ORDER BY date DESC LIMIT 10;	Nothing new for that indicator

Test 2 – deleting brings it back

#	Do this	Expect
1	Deviation list tab, tick the row, press Delete Deviation, confirm.	It moves to Deviation history

#	Do this	Expect
2	Download another flight.	The alarm is raised again

Test 3 – the date rule

This is the one that proves the logic rather than just "a deviation blocks alarms". A deviation must not affect a session that was flown before it existed.

#	Do this	Expect
1	Create a deviation again, as in Test 1.	The row turns green
2	Push its start date into the future: UPDATE public.indicator_deviation SET start_date = '2027-01-01' WHERE te_reference = 'BENO';	One row updated
3	Download a flight.	The alarm is raised, because every session now predates the deviation
4	Put the start date back to today and download again.	The alarm stops once more

Test 4 – nothing got slower

Watch the log line "Red/amber alarm analysis done in N ms" before and after. It should not grow noticeably: the deviations are fetched once per analysis, not once per threshold. If it jumps, the fetch has ended up inside a loop.

Test 1 is the headline – it is what REQ 12 asks for. Test 3 is what separates a working implementation from one that applies every deviation to every session regardless of when it was flown.

Two things to settle

Learned thresholds. As written, a deviation overrides the learned value as well as the configured one. That is the literal reading of the requirement, but it means a relearn that completes during the deviation is ignored until the deviation is closed. Confirm with Benoit.

More than one deviation per indicator. Step 3 keeps only the most recent. If an indicator can be deviated, acknowledged, then deviated again, a session dated between the two should use the first – which needs the map to hold a list. Worth asking whether that happens in practice before building it.